Proposal

by HERVE BIANCOTTO

Submission date: 16-Apr-2018 11:07PM (UTC+0100)
Submission ID: 86658209
File name: 101988_HERVE_BIANCOTTO_Proposal_442689_297684335.pdf
Word count: 5531
Character count: 29078

Deep learning for fixed income portolio optimisation

Project Proposal by Hervé Biancotto
MSc Data Science
Birkbeck, University of London

April 16, 2018

Contents

1 Background **3**
1.1 Advances in deep learning 3
1.2 Fixed income portfolio management 3
1.3 Deep learning for portfolio management 4

2 Literature review **5**

3 Objectives **7**

4 Method **8**
4.1 Design 8
4.2 Implementation 13
4.2.1 Database 13
4.2.2 Portfolio API 13
4.2.3 GUI 13
4.2.4 Deep learning models 14
4.3 Evaluation 14

5 Planning **15**

6 Outlook and further development **16**

7 References **17**

1 Background

1.1 Advances in deep learning

In recent years, deep learning has achieved tremendous results in fields such as speech recognition, computer vision and natural language processing (NLP), often superseding human capabilities. Approaches in predictive modelling have evolved from simple architectures - multi-layer perceptron (MLP) - associated with substantial feature engineering to architecture engineering whereby the structure of the model itself enhances predictive power and generalisation. The introduction of convolutional neural networks has led to major improvements in computer vision. Recurrent neural networks (RNN) and their subsequent derivations like Long Short-Term Memory (LSTM) have introduced long-term dependencies in the feature space, allowing models to leverage information that emanates from sequences or context. Residual blocks (ResNet and ResNeXt) have introduced a form of boosting to neural networks, allowing them to get deeper, i.e. to have more hidden layers, while avoiding typical optimisation issues like exploding or vanishing gradients. Thanks to the diversity of architectures, deep learning can tackle more complex optimisation tasks.

Though, despite state-of-the-art performance in some fields, deep learning models have been underutilised in analysis over tabular data and time series, mainly to the benefit of ensemble models or support vector machines. Yet, financial markets problems look particularly suited for deep learning. Data is abundant and available. Input can be a chart (vision task), time series (sequential task), cross-sectional data (regression task) or any combinations of those. The possibilities of deep learning for finance are yet to be explored. Within finance, many different fields can be considered. We will focus on portfolio management in the rest of this report.

1.2 Fixed income portfolio management

Traditionally, capital markets have been used by governments and companies to raise funding from the public. Bonds, a type of security, are fixed income instruments. Just like loans, they pay out interests, called a coupon, to holders over the duration of the contract between the borrower (the issuer or bond seller) and the lender (the creditor or bond buyer).

Most pension schemes and insurance companies invest in portfolios of stocks or bonds depending on their return target and their risk aversion. Typically, those portfolios are managed by asset management firms. The role of portfolio managers is to decide the allocation mix between assets within portfolios in order to realise clients' return objective under their risk tolerance.

According to the Bank for International Settlements (BIS) there were $92.2 trillion worth of bonds outstanding globally in 2016 whereas the world's stock

market capitalisation was only $70.1 trillion at that time, as reported by the World Federation of Exchanges.[1] Yet, quantitative research and academic literature generally deal with stocks or indices which are typically baskets of stocks or bonds. The main reason for this is the additional complexity to model bond trades. First, to compute the total return on a bond over a trading period, we need to accrue the income accumulated since purchase on top of the usual difference between the buying price and the selling price. Second, bonds also have an issuance date and a maturity date. In practice, it means that the history of prices is limited in time, both in the past and in the future. Lastly, a single company or government can issue multiple bonds at the same time. For example they can issue a 5 year, 10 year and 30 bond at the same time depending on the amount of funding they are after. Each bond will have its own price dynamics even though the issuing entity is the same across the three tranches. On the contrary, stocks have in general only one price series. Similarly, foreign exchange (FX) or currencies are more complex instruments than what intuition suggests. For example, if a fund manager wants to enter a trade whereby he/she sells (or short) Euro, taking a bearish view on the currency (i.e. the trade will win if the Euro depreciates vis-a-vis the US dollar), but the portfolio only has US Dollar in its bank account, the trade has to be implemented synthetically - through a forward contract for instance. Valuation of such trades are not straightforward.

1.3 Deep learning for portfolio management

In the 1970s, Eugene Fama stated that price formation followed the efficient market hypothesis (EMH). Within that framework, prices reflect all available information; past information or data have no influence over future prices, only new information matters. Later, in the 1980s, behavioural economics as popularised by Daniel Kahneman, emerged as an alternative to the EMH. Under this approach, investors have psychological, emotional and cognitive biases that result in behaviours which are not always consistent with what is expected from a Homo Economicus, a rationale agent that uses all information and knowledge to maximise his/her utility. Consequently, Kahneman concludes that humans are not skilled for portfolio management. Their biases impede them from outperforming naive portfolios. Empirical studies back Kahneman's conclusion. A review by S&P Dow Jones Indices found that only 5% of funds that have beaten their benchmark over three years managed to do so over the subsequent three years[2]. This presents an outstanding opportunity for automated portfolio management systems.

In practice, a toned-down version of the EMH, namely the weak form efficiency, is generally accepted by market practitioners. It states that prices only gradually reflect new information. This assumption explains the formation of price trends and reversals. Technical analysis, also known as charting, relies on

[1] SIFMA - 2017 fact book.

[2] *Active fund managers rarely beat their benchmarks year after year* by Tom Anderson, 27 Feb 2017, CNBC. Analysis over US large cap mutual funds benchmarked to the S&P index

the weak form efficiency hypothesis. It uses historical price charts in order to isolate repeating patterns that have some predictive power. For instance, technical analysts define trend channels and reversal patterns. Technical analysis seems to be a task computer vision models can tackle.

The key insight from technical analysis is that past price series contain valuable information to predict future prices. In other terms, prices exhibit a form of memory. It may be useful to consider the context around prices when it comes to forecasting future prices. In deep learning, gated architectures like LSTM have been able to introduce long term dependencies into models. By using LSTM, the model learns the context of the times series.

For a long time, signal processing techniques have been used to remove noise from financial time series in order to isolate frequencies and amplitudes or, in financial series terms, seasonality and trend magnitude. Interestingly, signal processing typically uses Fourier transforms to transpose information from time space into frequency space. In frequency space, it becomes easier to apply filters or convolutions to the data. Carefully selected convolutions manage to remove noise from the data. Up until deep learning, convolutions had to be designed by researchers. However, convolutions within a convolutional neural network (CNN) are parameters that get learnt by the model. Intuitively, it looks like CNNs can also be used to process financial time series.

Be it through CNNs or RNNs, deep learning has the potential to analyse financial series successfully while traditional portfolio managers struggle to deliver consistent results. In the rest of this report, I will propose an automated way to manage fixed income portfolios thanks to deep learning models. After outlining limitations and relevant findings of recent papers in section 2, section 3 presents the project's objectives, then section 4 describes the method. Section 5 introduces the development plan. The last section concludes with ideas for further development.

2 Literature review

Literature in quantitative finance have often been met with criticism. In their paper, Moallemi & Sağlam highlight the divide between academic research and practice. They attempt to avoid unrealistic assumptions about portfolio managers. They deem that past papers define unrealistic utility function for agents, while actual constrains come from exogeneous factors such as prohibition of short-selling or lack of access to liquidity - failing to find an opposite directional interest in the market to transact. They design a general linear rebalancing policy system that accounts for trading constraints. Yet, their approach only applies to trade execution - actual buying and selling of securities - not to portfolios. When looking at applying machine learning to quantitative finance,

Youngmin restricts his analysis to portfolios of stock when trying to correct for liquidity bias - assumption that liquidity is accessible at all time. Yang et al. portfolio optimisation technique also only applies to stocks. Petroupolos et al. introduce a machine learning based architecture to automate the optimisation of FX portfolios. In essence, they combine traditional machine learning techniques such as support vector machines (SVM), random forests (RF) and MLP to maximise portfolio's value. However, they do not apply trading techniques that portfolio managers use in practice. The model buys or sell currencies but fails to use forwards or options. The technical challenges to create an environment where research on fixed income portfolios can be conducted has created a shortfall in publication.

Beyond the lack of fixed income portfolio papers, very few articles have yet applied deep learning for portfolio management. Heaton et al. actually introduce replicating portfolios thanks to autoencoders. Autoencorders are neural networks that copy their input to their output thanks to an encoding process and a decoding process. Encoders and decoders are generally designed so that they learn useful features about the data as they intent to replicate them. This idea is particularly appealing for passive portfolio management. Passive portfolio management entails replicating an index so that investors get similar returns to that index without having to buy all underlying assets. It also happens that investors do not have access to the underlying assets; replicative portfolio gives them a synthetic exposure to those assets. The amount of tracking error, which is the standard deviation of the difference of returns between the portfolio and its index, determines the success of such a strategy. Even if passive management has been a growing market, active management - portfolio management that endeavours to outperform the return of an index thanks to successful allocation choices - plays a more crucial area for research. Indeed, if active management through deep learning is possible, passive management would, by definition, vanish. We will focus on active management going forward.

Given the lack of resources about deep learning for portfolio management, research on other subjects can be inspirational. We will retain three new ideas introduced by literature on deep learning. One early application of machine learning for retail banking and insurance companies was credit scoring. Kvamme et al. attempt to tackle this benchmark task with new architectures. They have applied convolutional neural networks, which are architectures that have traditionally been used in computer vision, to predict mortgage default probabilities. With a simple approach, the authors manage to obtain conclusive results, backing up the intuition that convolutional neural networks are adapted architectures for times series models. Recurrent networks have also traditionally been used to deal with times series. In order to facilitate the learning task, Wang & Tian improve recurrent neural networks by introducing residual blocks in the architecture. This is effectively implementing boosting on recurring networks, making them more stable to train. Lastly, Silver et al. associate deep networks

with reinforcement learning algorithm is order to derive optimal strategies to play the game of Go. All useful deep learning architectures are described in Goodfellow, Bengio and Courville text book, which will be the reference text book for network architectures in this project.

3 Objectives

The objective of the project is to apply deep learning models to manage active fixed income portfolios. This task can be decomposed in two distinct goals.

First, there is currently no robust back-testing environment for fixed income portfolios. Current fixed income portfolio representation systems either make simplifying - and unrealistic - assumptions so that computations are easy to perform or they are static - they are basically a collection of portfolio characteristics at a given date, for instance, the net asset value (NAV), but each representation is an independent snapshot. Conversely, the aim of a back-testing system it to allow the user to compose portfolios of tradable assets and to modify the asset mix or asset weights through time, basically simulating trading activity. Under that pattern, the system is able to assess the impact of trading strategies through time. Backtesting systems are crucial for portfolio optimisation tasks. The first objective of the project is to design such an environment.

The second goal is to optimise portfolios. For this project, I define optimising as finding the right set of assets and their associated weights through time so that the portfolio NAV is maximised. Yet, this definition is too general to base a strategy. Intuitively, the problem can be tackled either bottom-up or top-down. Under the bottom up approach, we focus on the objective function. Maximising NAV means maximising the weighted sum of individual asset values. Under that optimisation task, we could either select the assets with the highest return potential or put more weight on assets with a higher probability to realise projected returns. The objective is then reduced to predicting each asset potential return. This is a supervised learning task. On the other hand, we could adopt a top-down approach. The model starts from the entire investable universe of assets and narrows down the holdings to the set that maximises final NAV. This approach is interesting as the model should learn portfolio theory during training. It should learn that reducing the numbers of asset too much increases risk too much eventually leading to sub-optimal performance. In other terms, the model should learn how to diversify the portfolio. It should also learn how to leverage correlations between assets so that it picks high growth assets when correlations are low and selects low volatility assets when correlations trend up, which is typical in a crisis environment. However, trying to tackle this problem with brute force is nearly impossible. A portfolio typically contains 10% of its investable universe. For example, if the investable universe has 1000 assets, a typical portfolio has 100 assets. At each rebalancing period, there are 6.385×10^{139} possible combinaisons of 100 assets amongst

1000. It becomes even more complex if we consider that weights per assets can be tuned continuously, provided they add up to 100%. Fortunately, there is no need to design a *tabula rasa* system; we can use a priori information to reduce the feature space. In particular, it is unrealistic to consider the portfolio could rebalance the whole set of assets at each rebalancing date. The system will only trade at the margin from the latest available portfolio. Also trading has a cost that is reflected in assets' price series. If an asset is valued at price p_t at date t, the actual price paid for buying the security is $p_t + m_t$ while the price received for selling it is $p_t - m_t^*$ where m_t and m_t^* are positive numbers. The model should learn to reduce trading to its minimum in order to keep costs down. We want to achieve that level of knowledge through learning. We will adopt a reinforcement learning approach.

To summarise, the optimisation part has two objectives:

- Predict asset returns thanks to deep learning architectures.
- Define an optimal allocation/rebalancing policy so that portfolio returns are maximised.

4 Method

4.1 Design

Essentially, representing portfolios consists in:

- Constructing an adequate data structure for portfolios from raw asset data such as price series and other characteristics.
- Designing the API to interact with portfolios (compute and request NAV and risk metrics, buy or sell assets)

Initially, the available assets will be bonds and currencies. Currencies will be tradable through forward contracts in order to match what practitioners do. It implies the portfolio can go long and short currencies. On the other hand, we will forbid short-selling in bonds, as most real-life fund cannot short bonds. In practice, that means the weight of a bond will be positive. Lastly, there will be an asset that represents cash, i.e. the amount of money that is not invested.

Intuitively, a portfolio is a suitable concept for Object Oriented Programming (OOP). It is composed of assets, has certain attributes and we would like to perform some actions on them, like trading. However, assets, and by extension portfolios, have two types of attributes, either static or dynamic. Prices and weights are dynamic attributes; they evolve through time. On the other hand, the currency in which a bond is denominated does not change in time; it is constant. Due to this duality, designing an object model for assets is not

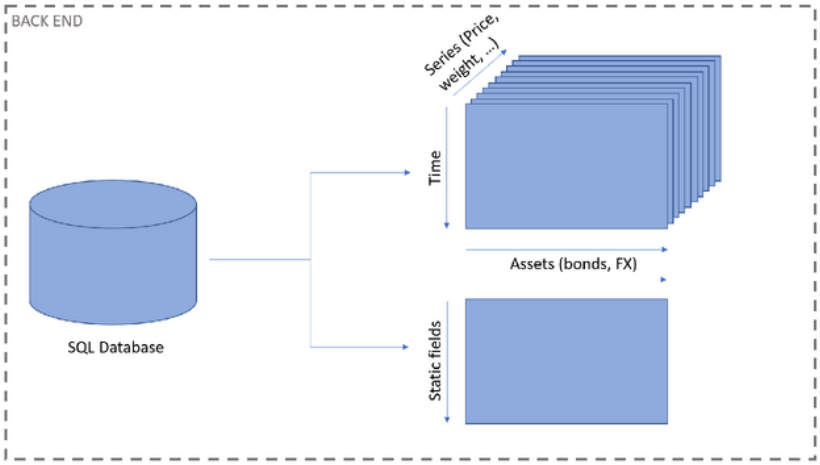

Figure 1: structure of the back end.

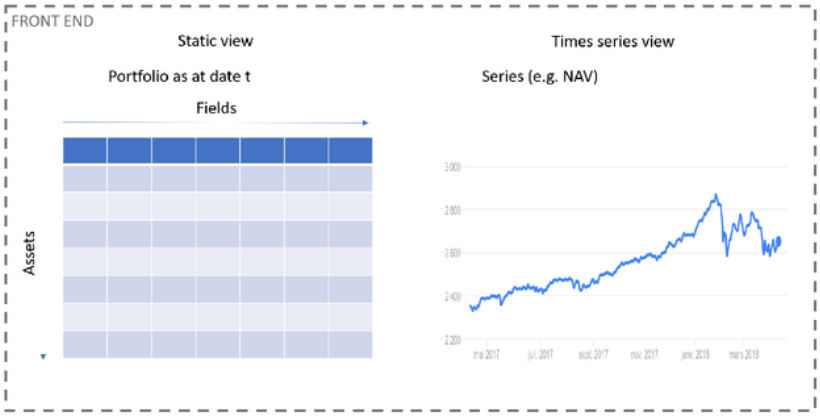

Figure 2: structure of the GUI.

trivial. As the actual goal of the API is to compute quickly the NAV of a portfolio through time, we also want to leverage matrix dot product. Under those constraints, we can render the portfolio as a collection of two-dimensional data tables. One table contains all static fields per asset in the portfolio. All other tables contain times series data for each asset. Each of these tables will be generated from raw asset data stored in a database. The structure of the whole back end is described in figure 1. I will define an API to allow users to interact easily with the back end. Once the back end is done, I will design a web-based GUI that use the API to retrieve and display portfolios. The structure of the front end is straightforward and can be seen on figure 2.

Forecasting asset returns is a supervised learning problem. As highlighted in the literature review, we will focus on three basic models: a CNN applied on historical price charts, a CNN applied to the time series and a LSTM model (please refer to figures 3 and 4). Models will produce two different outputs each, a classification task that assesses whether the price will be up or down, and a regression task that fits prices. The first output is a probability, the second output is a value. We will run those models on different time frequencies.
As per standard practice, the data will be split in three sets: training, validation and testing. It is important to keep data chronology when we validate the models. The validation set cannot be shuffled at random. Each validation set will be made of sub sets corresponding to the prediction time period, for example one week. The maximum prediction horizon will be three months. As a minimum, there will be twice as many data points as the prediction horizon between two consecutive validation subsets. In the three months case, there will be at least six months worth of data between two validation sub sets.
As far as inputs are concerned, chart images will have a standard format - constant width and height. The span of the chart will be a fixed time period so that predictions are generated from the same amount of data. Maximum and minimum points on the graph will be at fixed distance from the top and bottom of the image. This is to ensure all prices look on the same scale across assets. Similarly, for time series models (CNN and LSTM), continuous inputs (price and some engineered features) will be standardised using only the train set values to compute mean and standard deviation. This makes sure no knowledge of future distribution is used during learning. Standardisation makes it possible to compare predictions across assets. Categorical inputs like country or rating will be encoded thanks to embeddings. An embedding is a numerical vector representation of a categorical parameter. Emdeddings are learnt by the model.

Commonly-used architecture features will be implemented. CNNs and LSTM will implement batch normalisation (batchnorm) at every hidden level. This ensures stability during training. Again, batchnorm will only use train data for computations. CNNs will use adaptive max pooling in the last hidden layer prior to feed through the final fully-connected layer that outputs predictions. This is just to organise convolutional block's output so that they are inputs to

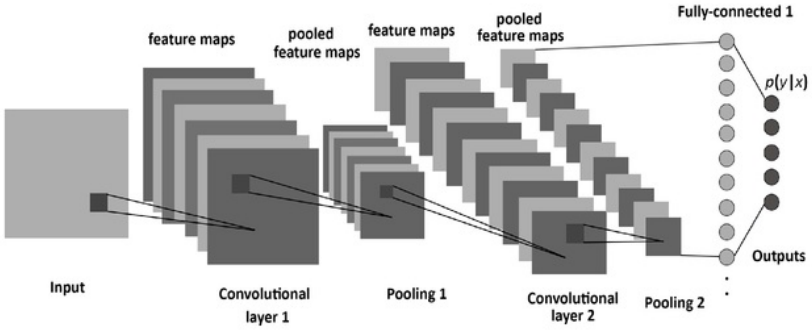

Figure 3: Convolutional neural network architecture.

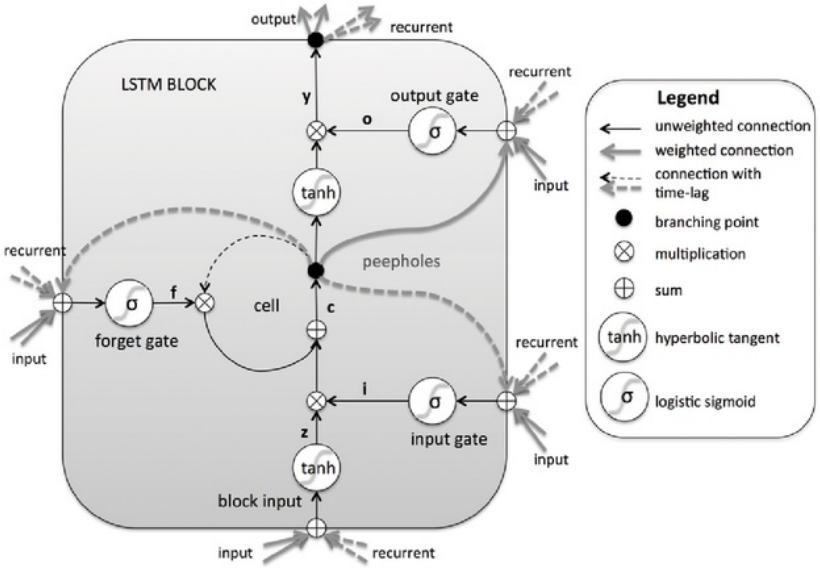

Figure 4: Long Short-Term Memory block in a RNN.

linear layers. Non-linearity for convolutional block will be the Rectified Linear Unit (ReLu). Non-linearity in the last layer will be the sigmoid function to output probabilities and a linear function to output regression results. There will be attempts to add residual blocks within CNNs and LTSM in order to introduce boosting; however, the actual architecture will depend on results.
From the CNNs and LSTM models, we obtain six predictions, three probabilities and three values. We will try out two ways to aggregate those values. Firstly, we will take a majority vote. Secondly, we will fit two shallow neural networks that takes the three probabilistic predictions and value predictions as input and that output a single probability and value as output. The weight will be learnt through fitting on the same training and validation set as the original three models. The method that yields the best aggregates will be retained.

The way to use the outputs from the previous models is not trivial. We will initially build a naive portfolio as a starting point. First, we create a prediction score which is simply the product of the projected probability by the absolute value of return prediction. We rank the scores and select the 100 best potential trades; we weight them uniformly.

It is clear that this naive approach is sub-optimal. To obtain optimal rebalancing policy, we would like to learn from our previous decisions and correct our errors through experience. This is an unsupervised task since we do not know what the optimal portfolio is. We will adopt a reinforcement learning approach. Reinforcement learning consists in consecutive trials and errors. The model gets rewarded if a good decision has been taken; it is penalised otherwise. The model learns through iterations.

The main challenge is to define the learning agent. In our case, the learning agent is the model that chooses whether to include an asset in the portfolio and what weight to assign to it. It also decides to sell an asset. In order to improve the decision mechanism, we introduce s_t to represent the state of an asset at time t. States will contain three criteria:

- Momentum: Exponential moving average of return
- Error: Exponential moving average of prediction error - difference between our prediction and the realised price.
- Diversification: Rolling correlation to the portfolio.

The goal will be for the agent to remember the choices of assets that led to successful allocation. In other words, it has to define which sort of states have to be selected and which one have to be avoided. At each allocation decision, a new portfolio will be created and compared to the current portfolio. If the new portfolio has a better return than the current version, the portfolio is rebalanced. There will be several control parameters to define how the agent will explore allocation possibilities. One important feature is that the portfolio will

be rebalanced at the margin. There is no need to explore the feature space with brute force. In practice, it is rare to rebalance more than 10 positions at the time; 0 to 5 positions is standard.

4.2 Implementation

The whole project will be written in Python. Python has all packages that make connections between different building blocks straightforward.

4.2.1 Database

The system will feed from data stored in a SQL database. SQL is well-adapted to table data. The database will be initially loaded from data available on Bloomberg. Bonds and FX securities will be the component of the *Bloomberg Barclays Global Aggregate Bond index* from 1 January 2001 to 31 December 2017, as well as the currency series of all non US Dollar currencies included in the index. There will be three price series per asset: the offer price represents the price that need to get paid to buy an asset; the bid price is the price that a seller receives to sell an asset; the mid price is the average between bid and offer prices. Having those 3 series ensures trading costs can be reflected. Other series will be engineered either descriptive (e.g. sector, rating, country) or engineered from price series. Finally, the user will be able to load and save portfolios from and to the database. The database will be stored locally in the development phase. Eventually, it will be stored on Amazon Web Service (AWS) Relational Database Service (RDS). As per python package, SQLAlchemy will handle the communication with the database.

4.2.2 Portfolio API

Computations and data manipulations at portfolio level will be based on Pandas. Pandas itself is built on Numpy, which will also be used. Each portfolio will be a collection of Pandas DataFrames (DF) (see blue rectangles in figure 1). The underlying assets are columns in those DF. Buying an asset is then equivalent to adding a column to all DF. Selling an asset however does not trigger a column deletion as the history of past holdings would be lost. Weight will have a 0 value. The cash asset weight will decrease or increase of the equivalent amount to the purchase or sale. Cash is bounded between 0% and 100%. All the above functionality will be packaged so that a user could work with portfolios in any python development environment by importing the package.

4.2.3 GUI

The GUI will use Dash in order to render portfolio views in a web browser. Dash is itself derived from Plotly and uses Flask as a web framework. The advantage of using Dash is that all the coding is made in Python. As well as this, Pandas and Dash interact easily. The GUI will be deployed on AWS Elastic Beanstalk, AWS cloud hosting service for applications.

4.2.4 Deep learning models

Deep learning models will be build in Pytorch. Pytorch has demonstrated high flexibility thanks to the use of dynamic computational graph. Common optimisers like stochastic gradient descent (SGD) or Adam are already implemented in Pytorch. Similarly, common loss function like cross-entropy loss or Mean Square Error (MSE) loss are also available. NN models will be improved by fine-tuning hyperparameters, for example, adding a learning rate scheduler, or by applying a different optimiser, for instance SGD with reset.

All optimisation task will be run on AWS Elastic Cloud Compute (EC2) P2 large instances to leverage the use of GPU. Models will be built in a Jupyter Notebook in order to facilitate experimentation as well as output visualisation and manipulation.

The training period will range between 1 January 2001 to 31 December 2011 (2,870 working days). Cross-validation will span over 20% of the training set (574 working days). The test set will be split in two. The first test set will be used for asset forecast modelling, from 1 January 2012 to 31 December 2015 (781 working days). It will then become the training set for the reinforcement learning model. The second test set will gauge how successful the allocation policy model is. It will run on the last three years of data (782 working days).

4.3 Evaluation

It is essential that the portfolio representation API is fully functional and reliable as the whole modelling task relies on it. Consequently, extra care will be taken when designing the system. I will use a test-driven development approach to make sure functionalities gets validated as building progresses. Also, as a final test, I will replicate an official bond index in the system by creating a portfolio with all the bonds that are in the index. The replicated index should output the same returns as the actual index. This will validate the portfolio representation API.

Careful scrutiny of how deep learning models evolve is crucial too. Models will be build and tested locally first on a reduced data sample, using only CPU. Only when the local version of the model successfully learns, which means the error rate drops substantially, will I run it on AWS EC2. The success of the optimisation task will be assessed by comparing the model portfolio to three benchmarks. The first one is the naive buy-and-hold approach, equivalent to buying the index. The second one will be the median of a group of portfolios managed by asset managers and benchmarked to the Bloomberg Barclays Global Aggregate Bond index. The third and last one will be the best performing portfolio within the previous group. The evaluation period will be set to three years as it is an industry standard. It will run from 1 January 2015

Figure 5: Time line of the project.

to 31 December 2017. Trivially, the success of the model is achieved through outperformance versus the benchmarks over that period.

5 Planning

There are some useful resources online for time management and project organisation. I will use Trello to manage tasks. It is a dashboard system where the user can create tasks and assess their progress. It is important to use a task manager in order to be able to quantify and assess the progress of the project as it goes. I will also time each task with Toggl. Toggl is simply an online timer that keeps track of user's task history. This will make sure I do not spend too much time on a single task and accumulate delays.

The project itself will be split in four blocks; each of them will span over a similar time horizon. The first one will tackle the creation of the database and portfolio API. The second block, building asset return models and creating a bottom-up portfolio, relies on the achievement of the first block. The third block implements the top-down approach with reinforcement learning. Lastly, I will put all the results together and deploy the GUI on AWS in order to make results accessible. The whole time line can be seen on the Gantt chart (figure 5).

6 Outlook and further development

The project can be a basis for further development. One topical portfolio management trend is to restrict the universe of securities in which portfolios can invest. For example, some management mandates excludes investments on the basi of Environmental, Social and Governance (ESG) criteria. A natural extension of the project is to add constraints to the optimisation task.
Another straightforward add-on to the project is to integrate more asset classes. The portfolio API can be extended to support extra assets like stocks or derivatives. The same optimisation procedure would then run on a multi-asset universe. Finally, all the optimisation task can be realised on different frequencies. This approach could be applied for high-frequency trading.

7 References

Literature:

Heaton, J. B., Polson N. G. & Witte J. H. (2017): *Deep learning for finance: deep portfolios*, Applied Stochastic Models in Business and Industry, John Wiley & Sons Ltd.

Wang, Y. & Tian, F. (2016): *Recurrent Residual Learning for Sequence Classification*, Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, Austin, Texas, USA, November 2016, Association of computational linguistics.

Moallemi C. C. & Sağlam M. (2017): *Dynamic Portfolio Choice with Linear Rebalancing Rules*, Journal of Financial and Quantitative Analysis, University of Washington, Seattle.

Youngmin H. (2017): *Machine learning in quantitative finance*, PhD thesis, University of Glasgow, UK, Glasgow Theses Service.

Kvamme H., Sellereite N., Aas K. & Sjursen S. (2018): *Predicting Mortgage Default using Convolutional Neural Networks*, Accepted manuscript, Expert Systems With Applications.

Petropoulos A. , Chatzis S. P., Siakoulis V. & Vlachogiannakis N. (2017): *A stacked generalization system for automated FOREX portfolio trading*, Expert Systems With Applications, Elsevier Ltd.

Yang P.-Y. , Lai Z.-R., Wu X. & Fang L. (2017): *Trend representation based log-density regularization system for portfolio optimization*, Pattern Recognition, Elsevier Ltd.

Goodfellow I., Bengio Y. & Courville A. (2016): *Deep Learning*, The MIT Press.

Silver D., Schrittwieser J., Simonyan K., Antonoglou I., Huang A., Guez A., Hubert T., Baker L., Lai M., Bolton A., Chen Y., Lillicrap T., Hui F., Sifre L., van den Driessche G., Graepel T. & Hassabis D. (2017): *Mastering the game of Go without human knowledge.* Nature.

Figures:

CNN:

Albelwi S. & Mahmood A. (2017): *A Framework for Designing the Architectures of Deep Convolutional Neural Networks.* Entropy.

LSTM block:
Zazo R., Lozano-Diez A., Gonzalez-Dominguez J., T. Toledano D. & Gonzalez-Rodriguez J. (2016): *Language Identification in Short Utterances Using Long Short-Term Memory (LSTM) Recurrent Neural Networks.* PLOS one.